UNIVERSITÀ DEGLI STUDI DI UDINE
hic sunt futura

DIPARTIMENTO DI SCIENZE MATEMATICHE, INFORMATICHE E FISICHE

PROGETTO DI LINGUAGGI E COMPILATORI

Parte 2 - Parziale

Francesco Bortuzzo **157430**
Leonardo Danelutti **158786**
Lorenzo Della Giustina **158854**

Anno Accademico 2024/2025

Indice

1. Scelte di design (Integrate nella definizione del linguaggio) 1
1.a. Modifiche alla sintassi concreta (Integrate nella definizione del linguaggio) 1
1.b. Risoluzione conflitti (Alcuni conflitti sono già stati risolti nella definizione del linguaggio; il conflitto 105 è scritto sotto) 1
2. Definizione del linguaggio: scelte di design e risoluzione dei conflitti 2
2.a. Tipi 2
2.a.I. Type system (MEGLIO SOTTO) 3
2.b. Variabili 3
2.c. Costanti 3
2.d. Loop 3
2.e. Condizionali 3
2.f. Routine 4
2.g. Commenti 4
2.h. Separatori 4
2.i. Risoluzione conflitti rimanenti 5
3. Definizione del lexer e del parser 5
3.a. Modifiche al lexer 5
3.b. Modifiche al Parser 5
4. TODO: 5
5. Definizione del Type system 6
5.a. Gerargia dei tipi 6
5.b. Regole formali per il type system 6
5.b.I. Espressioni 7
5.c. Statements 9
5.d. Dichiarazioni 9
6. Analisi Semantica Statica 10
6.a. Type-checker ed environment 10
6.b. Errori 10
6.b.I. Propagazione degli errori 10
6.b.II. Routine parziali 11
6.b.III. Dangling references 11
6.b.IV. Presenza del main 11
6.c. Casting 11
6.d. Espressioni left-hand side 12
6.e. Costanti 12
6.f. Altro che non so se è da dire 12
7. Generazione del Three Address Code (TAC) 12
Bibliografia 14

1. Scelte di design (Integrate nella definizione del linguaggio)

- Le dichiarazioni di variabili contengono obbligatoriamente l'assegnamento (motivazione: vedi pagina 3 punto 2 `implementazione-compilatori-imperativi.pdf`).
- Le costanti possono avere solo tipi primitivi (visto che le costanti non devono allocare spazio in memoria) (FIXME: questa è ovvia la toglierei).
- La stringa è considerata un tipo primitivo (vedi pagina 3 punto 3 `implementazione-compilatori-imperativi.pdf`).
- I due tipi di iterazione indeterminata sono derivabili dall'implementazione del loop come previsto dalla sintassi concreta di Blue.
- `fn` è syntax sugar di `fn()`. (FIXME: non è una nostra scelta, è una cosa di blue)

1.a. Modifiche alla sintassi concreta (Integrate nella definizione del linguaggio)

- Dalla sintassi concreta sono state tolte classi, pre post condizioni, etc. perchè non coerenti con quanto richiesto (FIXME: questa è ovvia la toglierei).
- Nelle function call no punto (ovviamente perchè non abbiamo le classi).
- Blue supporta solo commenti single line (con «–» o «==»). I commenti «==» sono documentazione per classi/funzioni ed è stata tolta dalla nostra implementazione perchè non richiesti. Inoltre sono stati aggiunti i commenti multi-line « ».
- Aggiunta keyword `return` nelle funzioni (al posto di avere i nomi delle variabili di ritorno che avrebbero richiesto di avere dichiarazioni senza assegnamento).
- Le funzioni tornano solo un tipo
- aggiunti i tipi `val`, `valres` e `ref` per i parametri delle funzioni.
- La keyword `routines` (che in Blue viene usata solo per la dichiarazione di routines nelle classi) viene usata anche all'interno del corpo delle funzioni in modo che ci possano essere dichiarazioni di routine annidate

1.b. Risoluzione conflitti (Alcuni conflitti sono già stati risolti nella definizione del linguaggio; il conflitto 105 è scritto sotto)

- Il conflitto state 105 LExp vs RExp non è un problema. Essendo le LExp sottoinsieme delle RExp basta scegliere sempre la Shift.

- Le dichiarazioni di funzioni, nell'ambiente più esterno, devono essere precedute dalla keyword `routines` similmente a quanto succede in Blue per le dichiarazioni delle routines nelle classi. Per evitare ambiguità tipo:

  ```
  var
    a: Integer := b
  main ...
  ```

  dove main potrebbe essere sia un nome di funzione che un nome di una variabile.
- Aggiunto `;` alla fine degli Statement (a parte If e Loop) e delle dichiarazioni di costanti e variabili per evitare ambiguità tipo:

  ```
  var
    a: Integer := b
  ( ...
  ```

  dove con un lookahead di 1 non è chiaro se «b (« è l'inizio di una chiamata di funzione oppure «(« è l'inizio di una LExp. Oppure ambiguità tipo:

  ```
  var
    a: Integer := b
  * ...
  ```

  dove «*» potrebbe essere sia un deref che una moltiplicazione. Per lo stesso motivo è stato aggiunto un «;» anche nell'exit dei loop.

2. Definizione del linguaggio: scelte di design e risoluzione dei conflitti

Il linguaggio definito per questo progetto è un linguaggio di programmazione imperativo con scoping statico. La sua grammatica si basa sulla sintassi concreta del linguaggio Blue.

Poiché Blue è un linguaggio orientato agli oggetti, per garantire le funzionalità richieste dalla consegna, come per esempio il dereferenziamento, sono stati introdotti alcuni elementi. Tuttavia, non sono presenti né classi né interfacce.

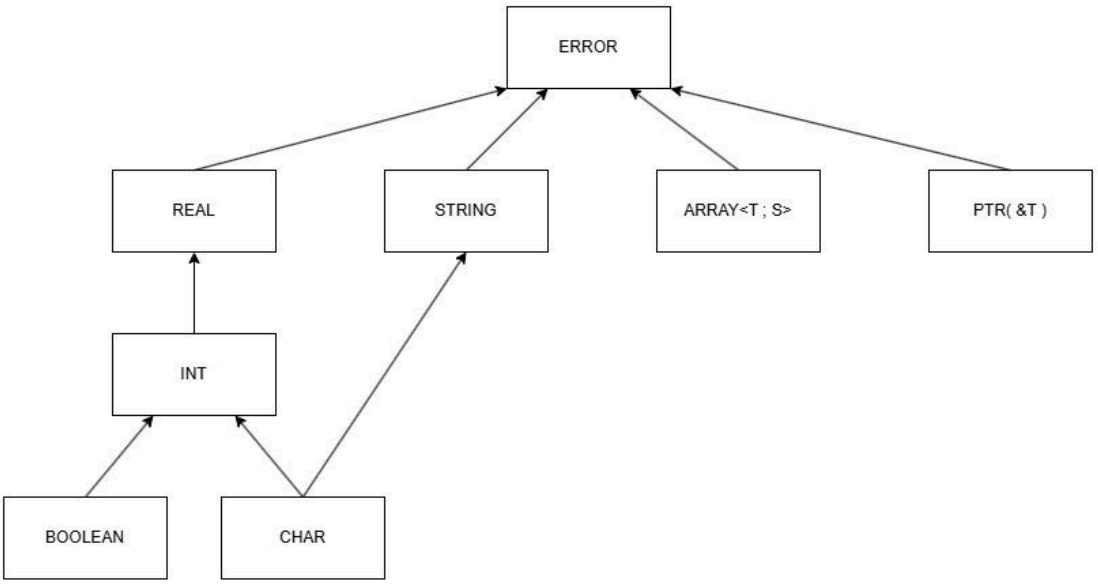
2.a. Tipi

Il linguaggio definito prevede i seguenti tipi primitivi: `Boolean`, `Integer`, `Real`, `String`[1] e `Char`. Sono inoltre supportati gli array che sono dichiarati utilizzando nel modo seguente: `Array<Type, Integer>`, dove `Type` rappresenta il tipo degli elementi e `Integer` indica la dimensione dell'array. Infine vi sono i puntatori, che sono dichiarati con la sintassi `&Type`.

[1] [1] M. Comini, «Front-end di un compilatore per un linguaggio imperativo: note generali», 2024., p. 3.3

2.a.i. Type system (MEGLIO SOTTO)

Per il nostro linguaggio, abbiamo definito la seguente gerarchia dei tipi:



2.b. Variabili

Le variabili sono dichiarate con la keyword `var` e possono essere di tipo primitivo o di tipo array. Non è consentito dichiarare variabili senza assegnamento[2].

2.c. Costanti

Le costanti sono dichiarate con la keyword `const` e possono essere di tipo primitivo (non devono allocare spazio in memoria). Anche in questo caso non è consentito dichiarare costanti senza assegnamento[3].

2.d. Loop

Per il nostro linguaggio sono previste le forme di iterazione non determinata `while-do` e `do-while`. Entrambe le strutture sono naturalmente implementate dall'elemento `loop-exit on` della sintassi concreta di Blue.

2.e. Condizionali

Il linguaggio immplementa le strutture condizionali semplici `if-then` e `if-then-else`, che permettono di eseguire blocchi di codice in base a condizioni specifiche.

[2] [1], p. 3.2
[3] [1], p. 3.2

2.f. Routine

Le routine comprendono sia funzioni che procedure. Una funzione senza parametri può essere invocata sia con la sintassi completa `function()` sia in forma abbreviata `function`, che è semplicemente zucchero sintattico per la prima. Rispetto alla sintassi concreta del linguaggio Blue, sono rimosse pre e post-condizioni; inoltre i parametri di ritorno non vengono nominati esplicitamente nella dichiarazione: si specificano solamente i tipi restituiti mediante la notazione `-> Type`. Si nota che non è possibile restituire tuple. Per gestire il ritorno esplicito dei valori, è stato introdotto il costrutto `return`. Le dichiarazioni di funzioni, in ogni ambiente, devono essere precedute dalla keyword `routines`, in modo simile a quanto avviene in Blue per le dichiarazioni delle routine nelle classi. Questo è necessario per evitare ambiguità, come nel seguente esempio:

```
Blue
var
  a: Integer := b
main ...
```

dove `main` potrebbe rappresentare sia il nome di una funzione che il nome di una variabile.

Quando si invoca una funzione è possibile specificare la modalità di passaggio dei parametri in ingresso (`val` , `valres` e `ref`). Infine, poiché il linguaggio non prevede il concetto di classi, le funzioni non possono essere invocate con la notazione a punto, come `.function()`.

2.g. Commenti

Il linguaggio supporta due tipi di commenti:

- Commenti a riga singola, introdotti dal simbolo `--`, e
- Commenti a blocco, delimitati da `/*` e `*/`.

2.h. Separatori

Al fine di disambiguare la grammatica e evitare conflitti è fondamentale l'introduzione di separatori alla fine degli Statement, salvo If e Loop, e delle dichiarazioni di costanti e variabili. Senza i separatori, infatti, il parser LALR(1) non sarebbe in grado di distinguere con certezza la struttura del codice in determinate situazioni.

Ad esempio:

```
var
  a: Integer := b
( ...
```

In questo caso, con un lookahead di 1 token, non è possibile determinare se `b (` rappresenti l'inizio di una chiamata a funzione oppure se `(` sia l'inizio di una Left Expression.

Un'altra ambiguità possibile è la seguente:

```
var
  a: Integer := b
* ...
```

dove `*` potrebbe essere sia un deref che una moltiplicazione. Per lo stesso motivo è stato aggiunto un `;` anche nell'exit dei loop.

2.i. Risoluzione conflitti rimanenti

La grammatica definita presenta un solo conflitto, che si verifica nello stato 104, tra le produzioni di `LExp` e `RExp`. Questo conflitto non crea problemi per l'analisi sintattica, dal momento che le `LExp` sono un sottoinsieme delle `RExp`. Pertanto, è sufficiente scegliere sempre l'azione di shift.

3. Definizione del lexer e del parser

Lexer e Parser sono stati realizzati modificando i file-prototipo generati automaticamente da BNFC, in modo tale da adattarli alle specifiche della consegna. La grammatica utilizzata per la costruzione di lexer e parser è quella presente nel file `grammar.cf` e genera il linguaggio descritto in precedenza.

3.a. Modifiche al lexer

- Oltre la posizione iniziale di un token è stata aggiunta anche la sua posizione finale, in modo da poter generare errori più precisi.

3.b. Modifiche al Parser

Il parser è stato modificato in modo da utilizzare la sintassi astratta da noi definita nel file `AbsSyn.hs` (a differenza di quella generata da BNFC). La struttura delle regole non è stata alterata.

4. TODO:

- [ ] disambiguare funzioni con lo stesso nome nel tac

5. Definizione del Type system

5.a. Gerargia dei tipi

Per il nostro linguaggio, abbiamo definito la seguente gerarchia dei tipi:

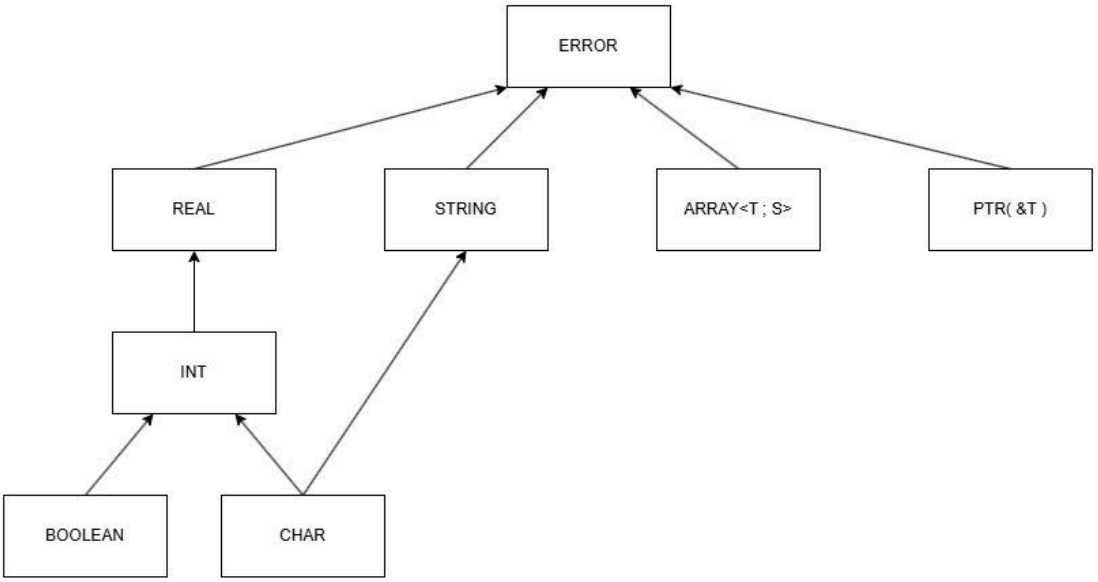


Figura 1: Gerarchia dei tipi.

5.b. Regole formali per il type system

Per definire il tipo di ogni nodo dell'albero di sintassi astratta, sono state definite delle regole formali. Queste regole sono utilizzate per determinare il tipo di espressioni, letterali e identificatori, in base all'ambiente corrente e ai tipi associati.

Di seguito sono riportate le regole formali utilizzate per definire tali tipi.

L'environemnt, oltre al suo normale scopo, viene usato paer propagare la necessità che un'espressione sia di tipo left-hand side. Nelle regole viene omesso, ma tutte le espressioni non-left-hand side tornano un errore se nel loro environment è presente la keyword `LExp` .

Inoltre, come spiegato nel paragrafo Sezione 6.a, nell'environemnt i namespace di simboli e routine sono separati. Per semplicità di scrittura nelle regole del type-system il namespace è

considerato come unico. Le eventuali ambiguità generano errori o sono gestite usando il contesto (in alcuni casi sono permesse solo left-hand side expression, e quindi non le chiamate a routine).

TODO: gestirle usando il tipo. non lo facciamo ancora

5.B.I. Espressioni

Di seguito le regole di annotazione delle espressioni. Si noti che i nodi interni propagano sempre gli errori senza greare «tipi composti da errori» per evitare errori annidati indotti da errori precedenti. Per semplicità di notazione tale processo è omesso nelle seguenti regole.

`LITERAL`

I literal sono foglie dell'AST. Il loro tipo dipende solo dal tipo di nodo.

$$\frac{}{\text{env} \vdash_E \text{Literal } \{\text{val} = \text{LitInt } \{\text{intVal} = e\}\} : \text{INT}} \tag{1}$$

$$\frac{}{\text{env} \vdash_E \text{Literal } \{\text{val} = \text{LitChar } \{\text{charVal} = e\}\} : \text{CHAR}} \tag{2}$$

$$\frac{}{\text{env} \vdash_E \text{Literal } \{\text{val} = \text{LitString } \{\text{stringVal} = e\}\} : \text{STRING}} \tag{3}$$

$$\frac{}{\text{env} \vdash_E \text{Literal } \{\text{val} = \text{LitBool } \{\text{boolVal} = e\}\} : \text{BOOL}} \tag{4}$$

$$\frac{}{\text{env} \vdash_E \text{Literal } \{\text{val} = \text{LitDouble } \{\text{doubleVal} = e\}\} : \text{REAL}} \tag{5}$$

`IDENT`

Anche gli ident sono foglie dell'AST. Possono essere:

- variabili;
- costanti;
- chiamate a funzioni senza parametri.

In tutti i casi assumono il tipo associato al loro nome nell'ambiente.

$$\frac{}{\text{env} \vdash_E \text{ExpIdent } \{\text{name} = \text{id}\} : \text{env}(\text{id})} \tag{6}$$

`FUNCALL`

Le chiamate di funzione prendono il tipo dall'environment. Gli argomenti supportano il casting implicito solo se passati per valore. Inoltre se usano un metodo di passaggio diverso da quello per valore devono essere espressioni left-hand side non costanti.

$$\frac{\text{env}(\text{id}) : \text{mode}_1\tau_1 \times ... \times \text{mode}_n\tau_n \vdash_E \tau \qquad \forall i = 1, ..., n \ \text{checkArg}(\text{env}, e_i, \tau_i', \tau_i, \text{mode}_i)}{\text{env} \vdash_E \text{FunCall } \{\text{routCallName} = \text{id}, \text{args} = [e_1, ..., e_n]\} : \text{env}(\text{id})} \tag{7}$$

dove:

$$\begin{gathered} \text{checkArg}(\text{env}, e, \tau', \tau, \text{mode}) \\ = \\ \text{env'} \vdash_E e : \tau' \\ \wedge \\ \text{mode} = \text{val} \rightarrow (\sup(\tau', \tau) = \tau \wedge \text{env'} = \text{env}) \\ \wedge \\ \text{mode} \neq \text{val} \rightarrow (\tau' = \tau \wedge \text{env'} = \text{env} \cup \text{"LExp"}) \end{gathered} \tag{8}$$

`SIGN`

Il segno prevede un'espressione numerica: un Integer o un Real. Siccome `sup(Integer, Real) = Real` (come definito in Figura 1), si prova prima a convertire `e` ad `Integer`:

$$\frac{\text{env} \vdash_E \text{exp} : \tau' \qquad \sup(\tau', \text{Integer}) = \text{Integer}}{\text{env} \vdash_E \text{Sign } \{e = \text{exp}\} : \text{Integer}} \tag{9}$$

Se `sup(tau', Integer) != Integer` si prova a convertire `e` a `Real`:

$$\frac{\text{env} \vdash_E \text{exp} : \tau' \qquad \sup(\tau', \text{Real}) = \text{Real} \wedge \sup(\tau', \text{Integer}) \neq \text{Integer}}{\text{env} \vdash_E \text{Sign } \{e = \text{exp}\} : \text{Real}} \tag{10}$$

BINOP

Le operazioni binarie supportate da `blue` si dividono in 4 categorie:

- le operazioni booleane:

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Boolean}) = \text{Boolean})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{AND}, \text{e1} = e_1, \text{e2} = e_2\} : \text{Boolean}} \tag{11}$$

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Boolean}) = \text{Boolean})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{OR}, \text{e1} = e_1, \text{e2} = e_2\} : \text{Boolean}} \tag{12}$$

- le operazioni aritmetiche intere:

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Integer}) = \text{Integer})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{DIVINT}, \text{e1} = e_1, \text{e2} = e_2\} : \text{Integer}} \tag{13}$$

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Integer}) = \text{Integer})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{MOD}, \text{e1} = e_1, \text{e2} = e_2\} : \text{Integer}} \tag{14}$$

- le operazioni aritmetiche reali:

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Real}) = \text{Real})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{DIV}, \text{e1} = e_1, \text{e2} = e_2\} : \text{Real}} \tag{15}$$

- le operazioni aritmetiche intere e reali:

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Real}) = \text{Real})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{ADD}, \text{e1} = e_1, \text{e2} = e_2\} : \sup(\sup(\tau_1, \tau_2), \text{Integer})} \tag{16}$$

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Real}) = \text{Real})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{SUB}, \text{e1} = e_1, \text{e2} = e_2\} : \sup(\sup(\tau_1, \tau_2), \text{Integer})} \tag{17}$$

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Real}) = \text{Real})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{MUL}, \text{e1} = e_1, \text{e2} = e_2\} : \sup(\sup(\tau_1, \tau_2), \text{Integer})} \tag{18}$$

$$\frac{\forall i = 1, 2(\text{env} \vdash_E e_i : \tau_i \wedge \sup(\tau_i, \text{Real}) = \text{Real})}{\text{env} \vdash_E \text{BinOp } \{\text{binOp} = \text{POW}, \text{e1} = e_1, \text{e2} = e_2\} : \sup(\sup(\tau_1, \tau_2), \text{Integer})} \tag{19}$$

BOOLNEG

La negazione prevede un'espressione booleana:

$$\frac{\text{env} \vdash_E \text{exp} : \tau \qquad \sup(\tau, \text{Boolean})}{\text{env} \vdash_E \text{BoolNeg } \{e = \text{exp}\} : \text{Boolean}} \tag{20}$$

RELOP

Le operazioni relazioni supportate sono quelle di uguaglianza e di confronto. Sono supportate da tutti i tipi, purchè convertibili allo stesso supertipo.

$$\frac{\text{env} \vdash_E e_1 : \tau_1 \qquad \text{env} \vdash_E e_2 : \tau_2 \qquad \sup(\tau_1, \tau_2) = \tau}{\text{env} \vdash_E \text{RelOp } \{\text{e1} = e_1, \text{e2} = e_2\} : \tau} \tag{21}$$

REF

È consentito creare reference soltanto di espressioni di tipo left-hand side.

$$\frac{\text{env' } \vdash_E \text{exp} : \tau \qquad \text{env'} = \text{env} \cup \text{"LExp"}}{\text{env} \vdash_E \text{Ref } \{e = \text{exp}\} : \tau} \tag{22}$$

FIXME: adesso le espressioni devono essere anche costanti per essere considerate LExp.

DEREF

Nei nodi dereference si controlla che l'espressione dereferenziata sia di tipo $\&\tau$.

$$\frac{\text{env} \vdash_E \text{exp} : \&\tau}{\text{env} \vdash_E \text{Deref } \{e = \text{exp}\} : \tau} \tag{23}$$

ARRAYINDEX

Nell'accesso agli array l'indice è vincolato ad essere un tipo convertibile ad intero, e l'array dev'essere un'espressione di tipo left-hand side.

FIXME: adesso per lo stesso bug di sopra dev'essere anche costante?

$$\frac{\text{env'} \vdash_E \text{exp} : \text{Array} < \tau, n > \wedge\ \text{env'} = \text{env} \cup \text{"LExp"} \qquad \text{env} \vdash_E \text{idx} : \tau' \wedge \sup(\tau', \text{Integer}) = \text{Integer}}{\text{env} \vdash_E \text{ArrayIndex } \{e = \text{exp}, \text{index} = \text{idx}\} : \tau} \tag{24}$$

ARRAY

Per creare un nodo array tutti gli elementi dell'array devono avere tipi compatibili tra loro. Il tipo finale contine la grandezza dell'array ed il least common ancestor tra tutti gli elementi.

$$\frac{\forall i = 1, ..., n(\text{env} \vdash_E e_i : \tau_i') \wedge \tau_1 = \tau_1' \wedge \forall i = 2, ..., n(\tau_i = \sup(\tau_{i-1}, \tau_i'))}{\text{env} \vdash_E \text{Array } \{\text{elements} = [e_1, ..., e_n]\} : \text{Array} < \tau_n, n >} \tag{25}$$

5.c. Statements

STMTASS

Negli assegnamenti viene controllato che:

- l'espressione a sinistra sia un'espressione left-hand side;
- l'espressione a destra possa essere convertita al tipo dell'espressione a sinistra.

$$\frac{\text{env} \vdash_S l : \tau_l \wedge \text{isLExp}(l) \wedge \text{isNotConst}(l) \qquad \text{env} \vdash_S r : \tau_r \qquad \sup(\tau_l, \tau_r) == \tau_l}{\text{env} \vdash_S \text{StmtAss } \{\text{left} = l, \text{right} = r\} : \emptyset} \tag{26}$$

STMTROUTCALL

TODO: finire

$$\frac{}{\text{env} \vdash_S \text{StmtRoutCall } \{\text{call} = \text{RoutineCall } \{\text{routCallName} = \text{id}, \text{args} = [e_1, ..., e_n]\}\} : \emptyset} \tag{27}$$

STMTRETRUN e **STMTRETRUNEMPTY**

Per la keyword `return` si controlla il tipo (se presente) che ha nell'environment.

$$\frac{\text{env} \vdash_E e : \tau \wedge \sup(\tau, \text{env}(\text{return})) == \text{env}(\text{return})}{\text{env} \vdash_S \text{StmtReturn } \{\text{valReturn} = e\} : \emptyset} \tag{28}$$

$$\frac{\text{env}(\text{return}) : \emptyset}{\text{env} \vdash_S \text{StmtReturnEmpty } \{\} : \emptyset} \tag{29}$$

STMTCONTINUE

Per la keyword `continue` si controlla che sia presente nell'environment.

$$\frac{\text{env}(\text{continue}) : \emptyset}{\text{env} \vdash_S \text{annStmtContinue } \{\} : \emptyset} \tag{30}$$

STMTIF e **STMTIFELSE**

$$(\text{env} \vdash_E e : \tau \wedge \sup(\tau, \text{Boolean}) == \text{Boolean}) \frac{\text{TODO: va aggiunto per s_1, s_n?}}{\text{env} \vdash_S \text{StmtIf } \{\text{cond} = e, \text{body} = [s_1, ..., s_n]\} : \emptyset} \tag{31}$$

$$(\text{env} \vdash_E e : \tau \wedge \sup(\tau, \text{Boolean}) == \text{Boolean}) \frac{\text{TODO: va aggiunto per s_t1, s_tn e s_f1, s_fn?}}{\text{env} \vdash_S \text{StmtIfElse } \{\text{cond} = e, \text{bodyT} = [s_{\text{t1}}, ..., s_{\text{tn}}]\ \text{bodyT} = [s_{\text{f1}}, ..., s_{\text{fn}}]\} : \emptyset} \tag{32}$$

STMTLOOP and **STMTLOOPEXIT**

5.d. Dichiarazioni

PROGRAMDECL

CONSTASS

VARASS

`FUNCTION` e `PROCEDURE` e `ROUTBODY`

6. Analisi Semantica Statica

L'analisi semantica statica è implementata nel modulo `StatSem`. Più nello specifico è diviso in 4 sotto-moduli: `Exp`, `Decl` e `Stmt` rispettivamente responsabili dell'annotazione delle espressioni, delle dichiarazione e degli statement, e `Utils` in cui sono definite utility e tipi di dato usati in tutto il modulo, tra cui le annotazioni, l'environment ed il casting.

6.a. Type-checker ed environment

L'annotazione dei tipi delle espressioni segue le regole del type system definito sopra.

Per poter annotare correttamente i tipi viene propagato un environment a partire dal nodo radice alle foglie (al contrario rispetto a come procede l'annotazione). Il namespace tra routine e simboli (variabili e costanti) è separato. Questo talvolta causa possibili problemi di ambiguità perchè le routine senza parametri possono essere chiamate anche senza `()` (scrivere `main` è zucchero sintattico per `main()`) come nella sintassi concreta di `blue`.

Inoltre è stato aggiunto il namespace `keyword` in cui, oltre a segnalare la possibilità di usare le keyword `return` e `continue`, si può richiedere che l'espressione sia di tipo left-hand side. Quindi quando un nodo viene annotato controlla nel proprio ambiente la necessità o meno di essere un'espressione left-hand side.

L'ambiente iniziale in cui è eseguito il programma contiene le funzioni predefinite.

TODO: merge di environemnt (probabilmente da mettere nel type system).

6.b. Errori

Il modulo di analisi semantica statica gestisce tutti gli errori a compile-time e garantisce al modulo di generazione del codice TAC un AST privo di errori.

6.b.i. Propagazione degli errori

Gli errori vengono propagati fino alla radice dell'AST. Se la radice non è priva di errori il codice non viene tradotto in TAC. Gli errori sono gestiti in due modi diversi in base al tipo di nodo:

- le espressioni che presentano errori hanno il tipo `TError`. In particolare può capitare che un'espressione abbia questo tipo in due casi:
  - c'è un errore nell'annotazione di quell'espressione. Quindi si crea un nuovo errore con un messaggio di errore opportuno;
  - c'è un errore in una o più sotto-espressioni. In questo caso si propagano gli errori (per questo il tipo `TError` ha una lista di `String`).

  TODO: dire come vengono evitati gli errori annidati indotti?

- gli statement e le dichiarazioni non hanno tipo. Per questto motivo nelle loro annotazioni c'è un campo `errs :: [String]` contenente gli errori generati durante l'annotazione del nodo in questione e dei sotto-nodi.

6.b.II. Routine parziali

In `blue` la definizione di routine parziali è vietata dal modulo di analisi semantica statica. In particolare viene controllato che per ogni flusso di esecuzione del `body` delle routine ci sia un `return _`.

A tal fine è presente un campo `alwaysReturn :: Bool` nelle annotazioni degli statement. Questo campo prende significato solo per statement annidati dentro funzioni, e, come suggerisce il nome:

- è vero se ritorna per ogni esecuzione dello statement. Ad esempio:

```
if true then
  return 1;
end if
```

- è falso altrimenti. Ad esempio se `a` è una variabile booleana:

```
if a then
  return 1;
end if
```

Chiaramente il controllo è conservativo: nell'esempio sopra se `a` assume sempre valore `true` per qualunque esecuzione, non è possibile saperlo durante l'analisi semantica statica.

L'annotazione di `alwaysReturn` procede dalle foglie fino alla radice dell'AST:

- le uniche foglie con `alwaysReturn = true` sono le keyword `return _` e `continue` (quest'ultima perchè è permesso ciclare all'infinito senza ritornare);
- i nodi interni dell'AST sono annotati in base a come controllano il flusso di esecuzione. TODO: magari è da scrivere nelle regole del type system

Per compilare correttamente il campo `alwaysReturn` è stato aggiunto un campo `canExit`. In generale questo campo è vero se lo statement può impedire l'esecuzione degli eventuali statement che lo seguono (allo stesso livello). Ci sono due «eccezioni»:

- il return ha `canExit = false` perchè ci interessano solo i flussi che non ritornano;
- il continue ha `canExit = false` perchè, come specificato sopra, `continue` è considerato come una foglia che ritorna.

TODO: unreachable. Forse è meglio toglierlo

6.b.III. Dangling references

Per evitare puntatori pendenti a variabili deallocate si è deciso di vietare il return di reference.

6.b.IV. Presenza del main

"Blue" esegue la routine `main`. In particolare dev'essere una procedura senza parametri in input. Se tale routine non è presente nella dichiarazione più esterna del programma, viene ritornato un opportuno errore.

6.c. Casting

In `blue` è permesso il casting implicito. Il modulo di analisi semantica statica lo rende esplicito per la successiva traduzione in codice TAC.

Per farlo è stata implementata una funzione di supertipo `sup` che, dati due tipi, ritorna il least common ancestor in Figura 1. Il casting da τ a τ' è supportato sse $\text{sup}(\tau, \tau') = \tau'$. Tale regola vale dappertutto dove il casting è supportato (si noti che il casting non è supportato per le modalità di passaggio per riferimento e per value-result, in quanto richiedono delle espressioni di tipo left-hand side).

Il casting delle costanti avviene durante l'analisi semantica statica per risparmiare tempo a run-time.

Il casting non è esteso ai tipi composti: ad esempio non è possibile convertire direttamente un puntatore a `Integer` ad un puntatore a `Real`.

6.d. Espressioni left-hand side

- distinzione LExp e RExp:
  - per le reference
  - i parametri passati con Ref e valres
  - left-hand side di assegnamenti e dichiarazioni (utile perchè le funzioni possono essere chiamate senza parentesi -> quindi il parser non da errore)

6.e. Costanti

calcolo di valori costanti dove possibile

6.f. Altro che non so se è da dire

- durante gli assegnamenti (e dichiarazioni) il numeri di L-expr e di R-expr deve essere lo stesso
- Il numero di argomenti della funzione deve essere giusto
- Controllare che nessuna costante sia a sinistra di un assegnamento O in un parametro ref o valres

7. Generazione del Three Address Code (TAC)

Oltre alle istruzioni viste a lezione sono state aggiunte le seguenti istruzioni:

- `ArrayGen addr size` per la creazione di un array di dimensione `size`.
- `Custom str` per la generazione di un istruzione personalizzata, viene utilizzata per l'istruzione `END_PROGRAM` che indica la fine del programma
- `Comment` per la generazione di commenti nel codice TAC.

Altro da dire:

- Le stringhe (letterali) vengono allocate in una sezione statica, e sono accessibili tramite un indirizzo statico.
- Le espressioni e i parametri vengono generate da sinistra a destra (perché più intuitivo di destra verso sinistra?). Invece gli assegnamenti multipli vengono trattati come zucchero sintattico di assegnamenti «su più linee», ad esempio:

```
x, y, z := 0, 1, 2;
```

risulta nello stesso codice TAC di:

```
x := 0;
y := 1;
z := 2;
```

- le modalità di passaggio sono gestite nel seguente modo:
  - Per valore: i parametri formali vengono copiati nei nomi dei parametri locali e quindi viene chiamata la funzione.
  - Per ref: prima della chiamata di funzione ai parametri locali viene assegnata la reference dei parametri formali, quindi viene chiamata la funzione. All'interno del corpo della funzione prima di ogni utilizzo di parametri locali viene generata un'istruzione di deref.
  - Per valres: prima della chiamata di funzione ai parametri locali viene assegnata la reference dei parametri formali, quindi viene chiamata la funzione. Prima del corpo della funzione i parametri locali vengono deferenziati in una copia locale (con un apice alla fine del nome). Prima di ogni `return` quindi viene assegnato il valore della copia locale al parametro locale deferenziato.
- Le funzioni di relazione tra stringhe e array sono generate direttamente nel TAC, con nomi che seguono la convenzione `string<relop>'` o `array<relop><type>'`, dove `<relop>` è l'operatore di relazione e `<type>` è il tipo intero dell'array.
- sia le espressioni booleane che le condizioni per il flow-control sono valutate con short-circuit usando il fall-trough

Il modulo di generazione del TAC (reperibile nella cartella `TAC`) utilizza una monade di stato per generare il codice, consentendo di mantenere traccia di informazioni di stato, quali:

- contatori per la creazione di temporanei, di label e di indirizzi statici
- lo stream di output per il codice generato (al contrario, come tutti gli altri stream)
- uno stack di stream di codice per la gestione di routine annidate
- uno stream per i dati statici (stringhe)
- Una lista di label che viene associata alla prossima istruzione
- Una lista di coppie di label, per la gestione degli exit on/continue all'interno dei loop (possibilmente annidati)
- Un insieme che tiene traccia quali funzioni di comparazione tra stringhe o array sono già state generate

Nel file `TAC/State.hs`, oltre alla definizione dello stato appena descritto sono presenti anche funzioni di supporto per la gestione dello stato.

Notazioni usate nel printer:

- Gli identificatori e nomi di sono annotati con la posizione nel file originale in questo modo: `<ident>@<line>:<column>`
- i commenti sono preceduti da `#`
- Le copie locali dei parametri passati per `valres` hanno il nome del parametro seguito da un apice (‘)

- le funzioni di relazione tra stringhe e array hanno un apice (“) alla fine del nome
- le variabili che puntano a dati statici sono precedute da `$`

Bibliografia

[1] M. Comini, «Front-end di un compilatore per un linguaggio imperativo: note generali», 2024.